National University of Computer and Emerging Sciences, Lahore Campus

Course:	**CP Lab**	**Code:**	**CL103**
Program:	**BS (Computer Science)**	**Semester:**	**Fall 2018**
Duration:	**150 minutes**	**T. Marks:**	**40**
Date:	**Tuesday 30-10-2018**	**Weight**	**30**
Section:	**A, B, C**	**Page(s):**	**2**
Exam:	**Lab midterm**		

Instructions/Notes:

- Use of the internet, notes, codes, lab manuals, and flash drives is strictly prohibited.
- Plagiarism will result in **F** grade in lab.
- Code must be **indented properly**, failure to comply will incur a penalty.

Question # 1: **(20 marks)**

Implement the function compress, the compresses a 2D array to size R x C, and compresses it to size (R-2) x (C-2). The compression is done as follows: taking a slot and its 9 neighboring cells, and calculate their average. Store the average in output.

```
Int**compress(int **input, int rows, int cols);
```

Note: The input array will have minimum 3 x 3 rows and columns.
The working of the function is explained in the diagram below:

Input (7x8):

2	3	2	3	4	5	7	8
2	2	1	8	3	4	5	2
3	4	5	7	2	1	4	6
9	1	2	3	1	5	3	4
7	8	8	8	9	4	6	65
7	6	11	2	1	3	5	6
3	4	5	66	7	1	9	0

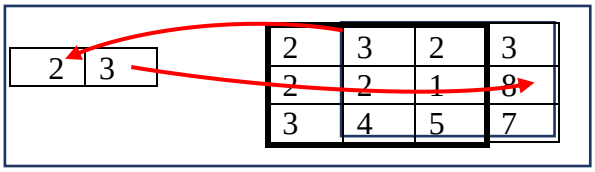
Output(5x6):

2					
				5	

For this you have to:

1. Write a function **int **Input (int &r, int &c);**
   which allocates the memory with r and c values after taking input from user.
2. Write a function **void output (int **matrix, int r, int c);**
   Which print the matrix.
3. Write a function **void deallocate (int **matrix, int c);**
   Which de-allocates the memory.

Call the Compress function which will return a compressed 2D array, you need to print that as well. And finally deallocate all the memory.



Here you can see, the input array is 3 x 4 and output array is 1 x 2.
Input (R x C) -> Output (R-2 x C-2).
In Output array, (first value is 2 which is average of first 9 elements and the value 3 is average of second 9 elements)

Question # 2: **(20 marks)**

In this question, you have to implement a HugeInt class that behaves like the int type, but can handle integers up to(maximum) 15 decimal digits. You will use dynamic arrays to store digits of various Huge Integers.

Define appropriate **constructors** *(Default,Parameterized and copy constructor)* and **destructor** required for this class.

Functions:

1. Arithmetic Function

   HugeInt addHugeInt (HugeInt obj2);
2. Function for comparing HugeInt numbers for

   bool isLessThan(HugeInt obj2)
3. Output Function for printing HugeInt number to the screen.
4. Input Function to take HugeInt number from the user.
   - Take input a HugeInt Number.
   - **Hint**: **For our ease,** you can take input in char array and then put every value in integer array.To take number from character array is like:

     char c = '5'; int num = c - 48;

Note: The integers can be positive only. Protype of HugeInt class and main function is given below:

```
class HugeInt{
        int *data, size;
public:
        void input();
        void output();
        HugeInt();
        HugeInt(int *d, int s);
        HugeInt(const HugeInt& obj);
        bool isLessThan(HugeInt& obj);
        HugeInt add(HugeInt obj);
        ~HugeInt()
};
```

```
void main()
{
        HugeInt obj1Int;
        obj1Int.input();
        int data[] = {1,2,3,4,5,6,7,8,9,4,3,1}, size = 12;
        HugeInt obj2Int(data,size);
        HugeInt obj3Int = obj1Int.add(obj2Int);
        obj3Int.output();
        cout<<" = ";
        obj1Int.output();
        cout<<" + ";
        obj2Int.output();
        cout<<endl;
        if(obj2Int.isLessThan(obj3Int)
        {
            cout<<"I am lesser\n";
        }
        else{
            cout<<"I am bigger\n";
        }
}
```

Output:

```
Input a huge number size: 4
Input a number: 5432
123456794863 = 5432 + 123456789431
I am lesser.
Press any key to continu...
```

National University of Computer and Emerging Sciences, Lahore Campus

Course:	**OOP Lab**	**Code:**	**CL217**
Program:	**BS (Computer Science)**	**Semester:**	**Summer 2019**
Duration:	**130 minutes**	**T. Marks:**	**100 (60+40)**
Date:	**Wednesday 10-07-2019**	**Weight**	**30%**
Section:	**A**	**Page(s):**	**3**
Exam:	**Lab Mid Term**	**Roll No:**	

Notes: No queries will be handled; plagiarism will be rewarded as 'F' grade.
Submission Path: \\cactus\Xeon\Summer 2019\Shakeel Zafar\Submissions\MID

Question # 1:

Complete the definition of the class given below such that the main program runs successfully. Make sure that your program doesn't consume extra memory space and it should not leak any memory.

```
class BinaryNum
{
private:
    int* binNum;          //integer array to save binary number
        int noOfBits;     //total no. of bits public:
 void Print(){   if(binNum != 0){
                          for(int i = 0 ; i< noOfBits ; i++)
                                    cout<<binNum[i];
                    }
                    cout<<endl;
        }
};
void main()
{
        BinaryNum b1;                       //noOfBits = 0, binNum is NULL
        BinaryNum b2("101");                //noOfBits = 3, binNum is {1,0,1}
        BinaryNum b3("1001");                //noOfBits = 4, binNum is {1,0,0,1}

        cout<<"b1 = ";cout<<b1;             //Prints Nothing
        cout<<"b2 = ";cout<<b2;             //Prints 101
        cout<<"b3 = ";cout<<b3;             //Prints 1001

        b1 = b2+b3;
        cout<<"b1 = "<<b1;                  //Prints 1110
        cout<<"b1[0] = "<<b1[0]<<endl;      //Prints 1 (0th bit in b1)
        cout<<"b1[3] = "<<b1[3]<<endl;      //Prints 0 (3rd bit in b1)

        cout<<b3++;                         //Prints 1001
        cout<<"is equal= "<<b3==b2;                 //Prints 0 cout<<++b3;
                                    //Prints 1011

        b1 = b2 + "111";
        cout<<b1;                           //Prints 1100
```

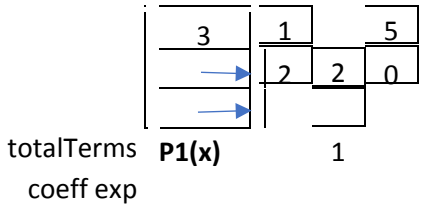
```
        cout<<b1-b2;                    //Prints 111

}
```

Question # 2:

A polynomial P1(x) = x^2+2x^1+5 has three terms: x^2, $2x^1$ and $5x^0$. Coefficients of these terms are 1, 2 and 5 respectively while exponents are 2, 1 and 0 respectively. You do not need to store exponent. Those will always be in decreasing order.

To work with Polynomials, a definition of class Polynomial is given below and memory configuration for P1 is shown as follows:

```
class Polynomial
{
private: int totalTerms;        //Total terms in a
Polynomial int* coeff;          //to save array of
coefficients };
```

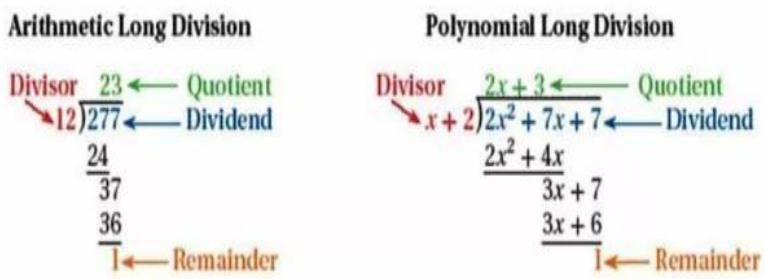


Your task is to complete the definition of Polynomial class such that the main program runs successfully. Make sure that your program doesn't consume extra memory space and it should not leak any memory.

```
void main()
{
        int coeff_P1[] = {1,2,5}, coeff_P2[] = {4,3};      //Coefficients for Polynomial P1 & P2 respectively

        Polynomial P1(3, coeff_P1);                  //Creates P1 with 3 terms (P1 = 1x^2 + 2x^1 + 5x^0 )
        Polynomial P2(2, coeff_P2);                  //Creates P2 with 2 terms (P2 = 4x^1 + 3x^0)

        cout<<"P1 = "; P1.Print();                   //Prints P1 = x^2+2x^1+5
        cout<<"P2 = "; P2.Print();                   //Prints P2 = 4x^1+3

        Polynomial P3 = P1*P2;                       //Multiplies P1 and P2 and saves result in P3. You may
                                                     //consume extra space for resultant Polynomial

        cout<<"P3 = "; P3.Print();                   //Prints P3 = 4x^3 + 11x^2 + 26x^1 + 15

        Polynomial P4 = P3/P2;                       //Prints P4 = x^2+2x^1+5
        cout<<"P4 = "; P4.Print();
}
```

P.T.O

For the above task you are given with following algorithm help.

Division Algorithm	**Multiplication Algorithm**
Step 0: Let we have A & B polynomial named for dividend and divisor respectively. We need two more polynomials named as quotient and remainder. **Step 1:** First arrange the term of dividend and the divisor in the decreasing order of their degrees. **Step 2:** To obtain the first term of quotient, divide the highest degree term of the dividend by the highest degree term of divisor. **Step 3:** To obtain the second term of the quotient, divide the highest degree term of the new dividend obtained as remainder by the highest degree term of the divisor. **Step 4:** Continue this process till the degree of remainder is less than the degree of divisor or remainder is zero.  	**Step 1:** Let we have polynomial A & B. Now, create a polynomial 'prod' which will have 'coeff' array of size m+n-1. **Step 2:** Initialize all entries in prod.coeff[] as 0. **Step 3:** Travers array A.coeff[] and do following for every element A.coeff[i] Traverse array B.coeff[] and do following for every element B.coeff[j] Prod.coeff[i+j] = prod.coeff[i+j] + A.coeff[i] * B.coeff[j]

National University of Computer and Emerging Sciences, Lahore Campus

Course:	**Object Oriented Programming Lab**	**Course Code:**	**CL217**
Program:	**BS (Computer Science)**	**Semester:**	**Fall 2020**
Duration:	**2 Hours**	**Total Marks:**	**100**
Paper Date:	**4-Nov-2020**	**Weight**	**30%**
Section:	**All**	**Page(s):**	**2**
Exam:	**Lab Midterm**	**Roll. No**	

Read below Instructions Carefully:

- Understanding the question statement is also part of the exam, so do not ask for any clarification. In case of any ambiguity, make suitable assumptions.
- You have to complete exam in 2 hrs. An extra 10 minutes are for submission.
- For Q1, create a folder named by your Roll number in the format 19L-9085 which should contain Matrix.cpp, Matrix.h and Driver.cpp. Submission Path is \\cactus\Xeon\Fall 2020\Hamna Waseem\OOP Labs\Midterm\BCS-3X\Q1. (X is your section name.)
- For Q2, submit a single file (containing function and main) named as 19L-9085.cpp on \\cactus\Xeon\Fall 2020\Hamna Waseem\OOP Labs\Midterm\BCS-3X\Q2. (X is your section name.)
- Submit both questions in a zipped folder (named as your roll number) on Slate in assignment titled as **OOP- Lab Midterm Submission**.
- Your code should be intended and commented properly. Use meaningful variable names.
- It is your responsibility to save your code from being copied. All matching codes will be considered cheating cases. PLAGIARISM will result in forwarding of case to Disciplinary Committee and negative marks in Midterm.

Question No. 01: **Marks: 60**

Any matrix can be broken down to a chunk of multiple sub-matrices. This process of splitting up the matrix into sub-matrices is called Matrix Partitioning.

For Example the matrix A is split into following 4 matrices.

$$A = \left[\begin{array}{ccccc|cc} 1 & 26 & 8 & 56 & 58 & 95 & 63 \\ \hline 3 & 4 & 56 & 89 & 53 & 102 & 36 \end{array}\right]$$

A1 = [1 26 8 56 58] A2 = [95 63]

A3 = [3 4 56 89 53] A4 = [102 36]

Matrix A has 2 rows and 7 columns that can be split horizontally and vertically into 4 sub matrices of different dimensions. In this example, the split is made on **1st row** and **5th column**. Even though there can be multiple splits in a matrix but we'll stick to four sub matrices for now.

Consider the following class that holds the 2-D matrix along with its dimensions.

```
Class my2D
{
private:
        int ** ptrto2D;
        int rows;
```

```
        int cols;
public:
        --------
};
```

Write a function with the following prototype that handles the partitioning of matrix:

my2D * SplitUp (const my2D & matrix);

The function returns the pointers pointing to the 4 partitioned matrices. The row and column from where to split will be taken as input from user in the function.

<u>A function is provided in function.txt file placed at \\cactus\Xeon\Fall 2020\Hamna Waseem\OOP Labs\Midterm that you might need for this questions' proper working.</u>

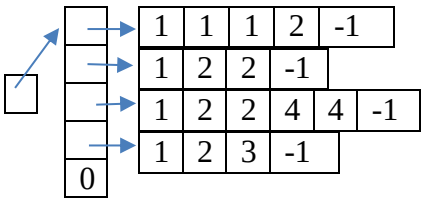
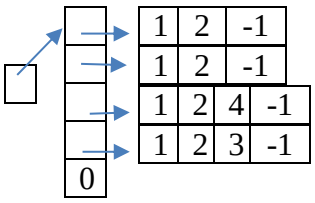
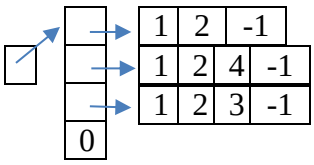
Your main program should do the following:

- Take input of matrix dimensions.
- Populate the matrix.
- Take input of rows and columns from where to split.
- Print actual matrix.
- Print the four partitions returned by function.

Your class my2D should have all the necessary function required to complete this task.

<u>Question No. 02:</u> **Marks: 40**

Write a function **void RemoveAllRepititions(int**& arr)** that takes a 2-D array "**arr**" as input and removes data repetitions in two passes. In first pass, it removes duplicate elements from each integer array. For example, integer array {1, 2, 2, 4, 4, -1} becomes {1, 2, 4, -1} after removing repetitions. In second pass, it removes repetitions if two consecutive arrays are identical. Sample run is shown below:

		
Input Array	**Array after Pass 1**	**Array after Pass 2**
	Identical consecutive elements have been removed from all rows	Identical consecutive rows removed. (First two integer arrays were identical in Pass 1)

Assume that elements in integer arrays are sorted in ascending order. Null (0) in int* array indicates end of integer arrays (delimiter in first dimension) while -1 indicates end of data in integer arrays (delimiter in second dimension). **Make sure that your program does not consume any extra memory and it should also not leak any memory.**

National University of Computer and Emerging Sciences, Lahore Campus

Course:	OOP Lab	Course Code:	CL1004
Program:	BS (Computer Science)	Semester:	Fall 2021
Duration:	2 Hours	Total Marks:	100
Paper Date:	20-Nov-2021	Weight:	
Section:	H	Page(s):	
Exam:	Lab Midterm	Reg. No.	

The word impossible itself says I'm Possible.

Read below Instructions Carefully:

- Understanding the question statement is also part of the exam, so do not ask for any clarification. In case of any ambiguity, make suitable assumptions.
- You have to complete exam in 2 hrs. No extra time will be given for submission.
- For Q1, Submit a single .cpp file named as 19L-9085-Q1.cpp
- For Q2, submit a single file (containing function and main) named as 19L-9085-Q2.cpp.
- Put both file in a folder, rename that folder with your roll number and paste/drop the folder at the mentioned: **\\Cactus\Xeon\Fall 2021\Dilawar Shabbir\MID Exam\Submission\Section H**
- Submit both questions .cpp on Google Classroom under assignment titled as **OOP- Lab Midterm Submission**.
- Your code should be intended and commented properly. Use meaningful variable names.
- It is your responsibility to save your code from being copied. All matching codes will be considered cheating cases. PLAGIARISM will result in forwarding of case to Disciplinary Committee and negative marks in Midterm.

Question No. 01:

Consider the following class that holds the student data.

```
class Student
{
   char *rollNo; //roll no of student like 20L-2383
   int semester; //semester number
   char *name; //name of student
   float CGPA;

// Your class Student should have all the necessary function required to complete the below task.

// You may add additional data members if required.


};

int main()
 {
```

```
Student S1,
S1.Input(); // Takes input from user.
S1.Output();  /*This function displays the Rollno,name,Cgpa and semester of the
calling Student object*/

Student list[5];
For(int I = 0; i<5 ; i++)
List[i].Input();

For(int I = 0; i<5 ; i++)
List[i].Output();

SortByCGPA(List);//Global function (Sorts the students on GPA base)

For(int I = 0; i<5 ; i++)
List[i].Output();

return 0;
}
```

Note: You cannot change the above main function.

Sample Output(working) on Console:

// Sample run is given for 2 students data.

```
Enter details of the students.
Enter Roll Number of the Student        : 21L-1122
Enter Current Semester of the Student   : 3
Enter Name of the Student               : Johnson Johnson
Enter CGPA of the Student               : 3.75

Enter details of the students.
Enter Roll Number of the Student        : 21L-1133
Enter Current Semester of the Student   : 3
Enter Name of the Student               : Paul Walker
Enter CGPA of the Student               : 3.57
```

// Once the data has been entered, your output function shows the details of the students.

```
Roll Number  : 21L-1122
Semester     : 3
Name         : Johnson Johnson
CGPA         : 3.75

Roll Number  : 21L-1133
Semester     : 3
```

CamScanner

Name : Paul Walker
CGPA : 3.57

// Now after calling of SortByCGPA function your data of students will be sorted in ascending order and will be shown as the student with least CGPA first and increasing down the list.

Roll Number : 21L-1133
Semester : 3
Name : Paul Walker
CGPA : 3.57

Roll Number : 21L-1122
Semester : 3
Name : Johnson Johnson
CGPA : 3.75

Question No. 02:

Factor calculator is a program which asks user how many inputs he wants to take and then asks user to enter **n** numbers such that, Numbers entered should be **greater than 6 and less than 100**. If the number does not fall in this range, program should alert user to re-enter the number. If the entered number is -1, the program should stop taking input and return the factors of all the entered numbers. Create factors array for each number in above allocated 2D array and store that number on 1st index of factors array and the factors on the further indexes. Your program should also **de-allocate** acquired dynamic memory at the end of the life cycle. **Do not consume** extra memory.

Int** GetFactorsArray(…) // This function takes count of elements and values of elements from user. Returns a dynamic 2-D Array of Factors of the elements(numbers) entered.

Void Print(…….); Displays 2-D array in proper format.

```
Void main()
{

// Design main as per your requirements.

}
```

Sample Output:

Enter no of inputs you want to take: 2

CamScanner

Enter Number between 6 and 100: 12

Enter Number between 6 and 100: 5

Wrong input. Please re-enter the number

Enter Number between 6 and 100: 10

...

...

Following are the factors of numbers entered:

Factors of 12 are 1,2,3,4,6,12
Factors of 10 are 1, 2,5,10

CamScanner

National University of Computer and Emerging Sciences, Lahore Campus

Course:	OOP LAB	Course Code:	CL1004
Program:	BSCS	Semester:	Spring 2022
Duration:	2 hours	Total Marks:	100
Paper Date:	13-April-2022	Weight	%
Section:	2K	Page(s):	3
Exam:	Midterm	Roll No.	

Read below Instructions Carefully:

- Understanding the question statement is also part of the exam, so do not ask for any clarification. In case of any ambiguity, make suitable assumptions.
- You have to complete exam in 2 hrs. Remaining time will be used for submission.
- For Q1, submit a single file (containing classes definitions and main) named as 21L-1122-Q1.cpp.
- For Q2, submit a single file (containing definitions and main) named as 21L-1122-Q2.cpp.
- Submission path: \\cactus1\Xeon\Spring 2022\Hira Ilyas\OOP Mid\2K
- Submit both questions .cpp file on Google Classroom under assignment titled as **OOP- Lab Mid Exam Submission. (Don't create zip file)**
- Your code should be intended and commented properly. Use meaningful variable names.
- It is your responsibility to save your code from being copied. All matching codes will be considered cheating cases. PLAGIARISM will result in forwarding of case to Disciplinary Committee and negative marks in Midterm.

Question No. 1

TASK 1:
Write a function **char** AllocateMemory(int& rows, int& cols)** that takes size of matrix (rows and columns) from the user , allocates memory for the matrix and return its pointer.

TASK 2:
Write a function **void InputMatrix(char** matrix, const int rows, const int cols)** which initializes the matrix by user input.

TASK 3:
Write a function **void DisplayMatrix(char** matrix, const int& rows, const int& cols)** that displays the matrix in proper format.

TASK 4:
Write a function **bool isAllDiagonal(char** matrix, const int& rows, const int& cols)** that returns true if every diagonal element has the same values in the matrix else false.

For Example:

Scanned with CamScanner

Input =

Output= True
Note: For isAllDiagonal the matrix should be square.

TASK 5:
Write a function **void rotateAntiClockWise(char** matrix, const int& rows, const int& cols)** that rotates the matrix in anticlockwise direction only if the matrix has all the diagonal values same.
For Example:
Input =

Output=

Question No. 02:

Define a class to represent a **Bank account**. Include the following members. Data members:-
Name of the depositor
Account number.
Type of account.
Balance amount in the account.

Provide a **default constructor**, a **parameterized constructor** and a **copy constructor** to this class.

Also provide Member Functions:-
1. To deposit an amount.
2. To withdraw the amount after checking for minimum balance.
3. To display all the details of an account holder.

Illustrate all the constructors as well as all the methods by defining objects.
In the main function, **dynamically allocate** an array of Bank Account objects. Your program should ask the user about the size of the array and then allocate the array dynamically. Then, your program should ask the user to enter the values of all attributes (Name of the depositor Account number, Type of account, Balance amount in the account) for each Bank Account. After that, your program should display the total number of Bank Account details as well as the details of each object on screen (by calling the display function for each Bank Account Object).

Scanned with CamScanner